

Deep Learning Lab6

312554041 謝博舟

1. Introduction

在本次實驗中，主要在實作兩種條件生成模型：Conditional GAN(生成對抗網絡)和 Conditional DDPM(擴散概率模型)。這些模型被設計來根據特定的 mult-label 條件生成精確的合成圖像。例如，當提出如“紅色球體”、“黃色立方體”和“灰色圓柱”等標籤時，模型需要能夠生成包含這些特定物體的圖像。而生成的圖像隨後將通過一個基於 ResNet18 架構的預訓練分類器進行評估，以確定圖像是否準確地反映了給定的標籤條件。

在 Conditional DDPM 的實作過程中，模型首先透過一個正向過程在原始圖像中增加噪聲，逐步達到完全的高斯分布狀態。這個增加噪聲的階段是模型學習生成圖像的基礎。接著，在反向過程中，模型逐步去除噪聲，最終產生無雜訊的、清晰的合成圖像。

同時，Conditional GAN 的訓練包含兩個互相競爭的神經網絡架構：Generator (生成器) 和 Discriminator (鑑別器)。Generator 會從給定的訓練數據中學習並嘗試創造新的、看似真實的圖像，而 Discriminator 則評估這些生成的圖像是否能夠被認為是來自於原始數據集的一部分。這一生成與鑑別的過程循環進行，直至 Discriminator 無法有效區分生成圖像和真實圖像，從而提高生成圖像的真實性和可信度。

2. Implementation details

a. DataLoader

```
class LoadDataset(Dataset):
    def __init__(self, root, mode, _transforms=None):

        self.root = root
        self.mode = mode
        self.transforms = _transforms if _transforms else self.default_transforms()

        if mode == 'train':
            with open('train.json', 'r') as json_file:
                self.json_data = json.load(json_file)
            self.img_paths = list(self.json_data.keys())
            self.labels = list(self.json_data.values())

        elif mode == 'test':
            with open('test.json', 'r') as json_file:
                self.json_data = json.load(json_file)
            self.labels = self.json_data

        elif mode == 'new_test':
            with open('new_test.json', 'r') as json_file:
                self.json_data = json.load(json_file)
            self.labels = self.json_data

        self.labels_one_hot = []
        with open('objects.json', 'r') as json_file:
            self.objects_dict = json.load(json_file)
            self.n_classes = len(self.objects_dict)

        for label in self.labels:
            label_one_hot = [0] * len(self.objects_dict)
            for text in label:
                label_one_hot[self.objects_dict[text]] = 1
            self.labels_one_hot.append(label_one_hot)
```

```

def __len__(self):
    return len(self.labels)

def __getitem__(self, index):
    if self.mode == 'train':
        img_path = os.path.join(self.root, self.img_paths[index])
        img = Image.open(img_path).convert("RGB")
        img = self.transforms(img)
        label_one_hot = torch.tensor(np.array(self.labels_one_hot[index]), dtype=torch.float32)
        return img, label_one_hot

    else:
        label_one_hot = torch.tensor(np.array(self.labels_one_hot[index]), dtype=torch.float32)
        return label_one_hot

def default_transforms(self):
    return transforms.Compose([
        transforms.Resize((64, 64)),
        transforms.ToTensor(),
        transforms.Normalize([0.5, 0.5, 0.5], [0.5, 0.5, 0.5])
    ])

```

```

def create_dataloader(root, batch_size, num_workers, mode, _transforms=None):
    dataset = LoadDataset(root=root, mode=mode, _transforms=_transforms)
    dataloader = DataLoader(dataset, batch_size=batch_size, shuffle=(mode == 'train'), num_workers=num_workers)
    n_classes = dataset.n_classes # 24
    return dataloader, n_classes

```

以上程式 dataloader.py 主要包含了一個用於加載和處理數據集的 Class LoadDataset 以及一個用於創建數據加載器 create_dataloader 的功能函數。其中依照訓練及測試的模式，分成 train, test 和 new_test 三個資料集，如果模式是 train，則加載和轉換圖像，並返回圖像及其對應的 one-hot labels，而對於 test 和 new_test，只返回 one-hot labels。

b. Conditional DDPM

這個實驗的 DDPM 主架構分成 UNet, DDPM 結構，以及 Train-Test 三個部份，將 mult-label 和 Timestep 當作條件並加入模型裡，並透過 UNet 架構的原理，將 input 的圖像和 condition 經由 DownSample 和 UpSample 控制想要保留的資訊並生成出圖片，下面會詳細介紹每個部份的原理和架構。

1. UNet 架構

```

class ResidualConvBlock(nn.Module):
    def __init__(self, in_channels, out_channels, is_residual=False):
        super().__init__()

        self.same_channels = (in_channels==out_channels)
        self.is_residual = is_residual

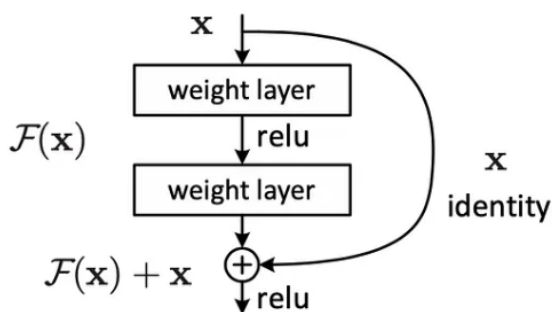
        # conv block
        self.conv1 = nn.Sequential(
            nn.Conv2d(in_channels, out_channels, 3, 1, 1),
            nn.BatchNorm2d(out_channels),
            nn.GELU(),
        )
        self.conv2 = nn.Sequential(
            nn.Conv2d(out_channels, out_channels, 3, 1, 1),
            nn.BatchNorm2d(out_channels),
            nn.GELU(),
        )

```

```
def forward(self, x: torch.Tensor) -> torch.Tensor:
    if self.is_residual:
        x1 = self.conv1(x)
        x2 = self.conv2(x1)
        # adds on correct residual in case channels have increased
        if self.same_channels:
            out = x + x2
        else:
            out = x1 + x2
        return out / 1.414
    else:
        x1 = self.conv1(x)
        x2 = self.conv2(x1)
        return x2
```

ResidualConvBlock 建構了一個基本的 ResNet 架構，包含兩個卷積層，每個層都進行了卷積、BatchNorm2d 和 GELU。最終輸出的是第二個卷積層的結果。

如果輸入和輸出通道數相同(`self.same_channels`)，則將原始輸入 x 直接與第二個卷積塊的輸出 x_2 相加，這樣可以直接將輸入特徵與學習到的特徵結合，如下面的圖：



如果輸入和輸出通道數不同，由於直接相加會導致維度不匹配，這時會將第一次卷積的輸出 x_1 與第二次的輸出 x_2 相加。

```
# down sampling image feature maps
class DownSample(nn.Module):
    def __init__(self, in_channels, out_channels):
        super(DownSample, self).__init__()

        self.model = nn.Sequential(
            ResidualConvBlock(in_channels, out_channels),
            nn.MaxPool2d(2)
        )

    def forward(self, x):
        out = self.model(x)
        return out

# up sampling image feature maps
class UpSample(nn.Module):
    def __init__(self, in_channels, out_channels):
        super(UpSample, self).__init__()

        self.model = nn.Sequential(
            nn.ConvTranspose2d(in_channels, out_channels, 2, 2),
            ResidualConvBlock(out_channels, out_channels),
            ResidualConvBlock(out_channels, out_channels)
        )

    def forward(self, x, skip):
        out = torch.cat((x, skip), 1)
        out = self.model(out)
        return out
```

DownSample 包含 ResidualConvBlock 和一個 MaxPool2d 組成的模型，在 forward 中，輸入的圖像透過序列模型進行處理，實現連續的捲積和池化操作，輸出降採樣後的特徵圖。

UpSample 包含 ConvTranspose2d 和兩個 ResidualConvBlock 的序列模型，ConvTranspose2d 用於將特徵圖的尺寸加倍，從而逐步恢復到原始影像的尺寸。跟隨的 ResidualConvBlock 進一步處理擴大後的特徵圖，增強特徵同時保持尺寸不變。在 forward 中，將輸入的特徵圖與跳躍連接的特徵圖(skip)在通道維度上進行拼接。

```
class Embed(nn.Module):
    def __init__(self, input_dim, emb_dim):
        super(Embed, self).__init__()

        self.input_dim = input_dim
        self.model = nn.Sequential(
            nn.Linear(input_dim, emb_dim),
            nn.GELU(),
            nn.Linear(emb_dim, emb_dim)
        )

    def forward(self, x):
        out = x.view(-1, self.input_dim)
        out = self.model(out)
        return out
```

Embed 輸入資料經過連續的線性變換和 GELU 處理，最終輸出一個維度為 emb_dim 的 embedding vector。並且在 forward 中處理輸入 x (使用 view 將輸入 x 調整為 (-1, self.input_dim) 形式)，確保無論輸入的批量大小如何，每個樣本都被平展為一個長度為 input_dim 的一維向量。


```

# Unet model
class UNet(nn.Module):
    def __init__(self, in_channels, n_feature=256, n_classes=24):
        super(UNet, self).__init__()

        self.in_channels = in_channels
        self.n_feature = n_feature
        self.n_classes = n_classes
        self.initial_conv = ResidualConvBlock(in_channels, n_feature, is_residual=True)

        self.down1 = DownSample(n_feature, n_feature)
        self.down2 = DownSample(n_feature, 2 * n_feature)

        self.hidden = nn.Sequential(nn.AvgPool2d(8), nn.GELU())

        self.time_embed1 = Embed(1, 2*n_feature)
        self.time_embed2 = Embed(1, 1*n_feature)
        self.cond_embed1 = Embed(n_classes, 2*n_feature)
        self.cond_embed2 = Embed(n_classes, 1*n_feature)

        self.time_embed_down1 = Embed(1, 1*n_feature)
        self.cond_embed_down1 = Embed(n_classes, 1*n_feature)
        self.time_embed_down2 = Embed(1, 1*n_feature)
        self.cond_embed_down2 = Embed(n_classes, 1*n_feature)

        self.up0 = nn.Sequential(
            nn.ConvTranspose2d(2 * n_feature, 2 * n_feature, 8, 8),
            nn.GroupNorm(8, 2 * n_feature),
            nn.ReLU(),
        )
        self.up1 = UpSample(4 * n_feature, n_feature)
        self.up2 = UpSample(2 * n_feature, n_feature)

        self.out = nn.Sequential(
            nn.Conv2d(2 * n_feature, n_feature, 3, 1, 1),
            nn.GroupNorm(8, n_feature),
            nn.ReLU(),
            nn.Conv2d(n_feature, self.in_channels, 3, 1, 1),
        )

```

UNet 為主架構，模型呈現 U 形狀，結合了 ResidualConvBlock、DownSample、UpSample 以及 Embed。

```

def forward(self, x, cond, time):
    # embed context, time step
    cond_emb1 = self.cond_embed1(cond).view(-1, self.n_feature * 2, 1, 1) # [32,512,1,1]
    time_emb1 = self.time_embed1(time).view(-1, self.n_feature * 2, 1, 1) # [32,512,1,1]
    cond_emb2 = self.cond_embed2(cond).view(-1, self.n_feature, 1, 1) # [32,256,1,1]
    time_emb2 = self.time_embed2(time).view(-1, self.n_feature, 1, 1) # [32,256,1,1]
    # for down
    cond_emb_down1 = self.cond_embed_down1(cond).view(-1, self.n_feature, 1, 1)
    time_emb_down1 = self.time_embed_down1(time).view(-1, self.n_feature, 1, 1)
    cond_emb_down2 = self.cond_embed_down2(cond).view(-1, self.n_feature, 1, 1)
    time_emb_down2 = self.time_embed_down2(time).view(-1, self.n_feature, 1, 1)

    x = self.initial_conv(x) # [32,256,64,64]

    # down sampling
    down1 = self.down1(cond_emb_down1*x+ time_emb_down1)
    down2 = self.down2(cond_emb_down2*down1+ time_emb_down2)

    # hidden
    hidden = self.hidden(down2)
    up1 = self.up0(hidden) # [32,256,64,64]
    up2 = self.up1(cond_emb1*up1+ time_emb1, down2)
    up3 = self.up2(cond_emb2*up2+ time_emb2, down1)

    # output
    out = self.out(torch.cat((up3, x), 1))
    return out

```

在前向傳播中，首輸入 x 先透過初始的殘差卷積塊處理，然後經過兩個 DownSample，每一 DownSample 前都會將 `cond_embed` 和 `time_embed` 與特徵圖結合，以引導特徵提取過程。而中間會在最小尺寸的特徵圖上進行一次平均池化和激活，形成一個全局特徵表示。

再來，透過 UpSample 逐步恢復特徵圖尺寸，每一步都融合從對應 DownSample 傳遞來的特徵 (skip connection) 和 `cond`, `time_embed` 的特徵，以精細地恢復影像細節。最終將 UpSample 結果與原始輸入經由殘差連接合併後透過輸出層處理，得到最終的影像或特徵映射。

```

def build_model(args, n_classes, device, mode):
    if mode == 'train':
        print("\nBuilding models for training...")
        model = UNet(in_channels=args.in_channels, n_feature=args.n_feature, n_classes=n_classes)
        model.initialize_weights()
        return model.to(device)

    elif mode == 'test':
        print("\nBuilding models for testing...")
        model_test = UNet(in_channels=args.in_channels, n_feature=args.n_feature, n_classes=n_classes)
        model_test.initialize_weights()
        model_test_new = UNet(in_channels=args.in_channels, n_feature=args.n_feature, n_classes=n_classes)
        model_test_new.initialize_weights()
        return model_test.to(device), model_test_new.to(device)

```

`build_model` 目的是在根據不同的使用情景 (train 或 test) 來建構和初始化 UNet 模型。這個函數依據不同的模式 (訓練模式和 測試模式)，分別調整 UNet 架構的配置，每個模型都會經過權重初始化，而之後也會被轉移到 GPU 上，以便進行優化模型。

2. DDPM 架構

```
class ConditionalDDPM(nn.Module):
    def __init__(self, unet_model, betas, noise_steps, device, noise_type='linear'):
        super(ConditionalDDPM, self).__init__()

        self.n_T = noise_steps
        self.device = device
        self.unet_model = unet_model
        self.noise_type = noise_type

        if noise_type == 'linear':
            self.betas = torch.linspace(betas[0], betas[1], noise_steps).to(device)
        elif noise_type == 'cosine':
            timesteps = torch.linspace(0, 1, noise_steps)
            self.betas = (betas[0] + (betas[1] - betas[0]) * (1 + torch.cos(np.pi * timesteps)) / 2).to(device)
        else:
            raise ValueError("Unsupported noise type. Choose 'linear' or 'cosine'.")

        self.alphas = 1.0 - self.betas
        self.alphas_cumprod = torch.cumprod(self.alphas, dim=0).to(device)
        self.alphas_cumprod_prev = torch.cat([torch.tensor([1.0], device=device), self.alphas_cumprod[:-1]])

        self.sqrt_alphas_cumprod = torch.sqrt(self.alphas_cumprod).to(device)
        self.sqrt_one_minus_alphas_cumprod = torch.sqrt(1 - self.alphas_cumprod).to(device)
        self.sqrt_recip_alphas = torch.sqrt(1.0 / self.alphas).to(device)
        self.one_minus_alphas_cumprod = (1 - self.alphas_cumprod).to(device)

        self.mab_over_sqrtmab = (self.betas / self.sqrt_one_minus_alphas_cumprod).to(device)

        self.mse_loss = nn.MSELoss()

        self.device = device
```

在 ConditionalDDPM 的初始化方法中，主要負責設定模型所需的各項參數和計算相關係數。該方法的核心部分是設置 noise scheduler，決定了在模擬擴散過程中噪聲的變化方式。

可以選擇的噪聲調度方式有：

- 線性 (linear): 在給定的時間步長內，betas 值從 betas[0] 線性增長至 betas[1] (在這次實驗中，我設 betas 分別為 0.02 和 0.0001)。
- 餘弦 (cosine): 使用餘弦函數進行更平滑的過渡，用來改善加入 noise 過快的問題，betas 的值根據時間步長的餘弦變化計算，從 betas[0] 到 betas[1]，公式如下：

$$\bar{\alpha}_t = \frac{f(t)}{f(0)}, f(t) = \cos\left(\frac{t/T + s}{1 + s} \cdot \frac{\pi}{2}\right)^2 \quad (t: \text{step}, T: \text{Total step}, S = \sqrt{\beta})$$

接下來，根據計算出的 betas 值，以便在訓練和生成過程中使用：

- alphas: 表示每一步保留原始圖像信息的程度，計算為 1 - betas。
- alphas_cumprod: 計算所有前步 alphas 值的累積乘積，表示在當前步驟後，圖像中保留的原始信息量。
- alphas_cumprod_prev: 調整 alphas_cumprod 以獲得前一步的累積值。
- 其他如 sqrt_alphas_cumprod, sqrt_one_minus_alphas_cumprod, sqrt_recip_alphas, 和 one_minus_alphas_cumprod 等均為計算過程中用於方便 reverse process 的數學表示。

```
def forward(self, x, cond):
    """training ddpm, sample time and noise randomly (return loss)"""
    batch_size = x.shape[0]
    timestep = torch.randint(0, self.n_T, (x.shape[0],)).to(self.device)
    noise = torch.randn_like(x)

    sqrt_alphas_cumprod_t = self.sqrt_alphas_cumprod[timestep].view(batch_size, 1, 1, 1)
    sqrt_one_minus_alphas_cumprod_t = self.sqrt_one_minus_alphas_cumprod[timestep].view(batch_size, 1, 1, 1)
    x_t = sqrt_alphas_cumprod_t * x + sqrt_one_minus_alphas_cumprod_t * noise
    predict_noise = self.unet_model(x_t, cond, timestep / self.n_T)
    mse_loss = self.mse_loss(noise, predict_noise)

    return mse_loss
```

forward 函數在計算圖像通過模型後的重建損失，並使用公式來進行向量的計算，以下是所搭配的公式：

$$q(\mathbf{x}_{1:T}|\mathbf{x}_0) := \prod_{t=1}^T q(\mathbf{x}_t|\mathbf{x}_{t-1}), \quad q(\mathbf{x}_t|\mathbf{x}_{t-1}) := \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t}\mathbf{x}_{t-1}, \beta_t\mathbf{I})$$

$$\begin{aligned} x_t &= \sqrt{\alpha_t}x_{t-1} + \sqrt{1 - \alpha_t}z_{t-1} & x_t &= \sqrt{\bar{\alpha}}x_0 + \sqrt{1 - \bar{\alpha}}z \\ x_{t-1} &= \sqrt{\alpha_{t-1}}x_{t-2} + \sqrt{1 - \alpha_{t-1}}z_{t-2} \end{aligned}$$

其中： $\bar{\alpha}_t := \prod_{s=1}^t \alpha_s$

它首先隨機從 1000 個 noise step 中選擇一個 timestep，對輸入圖像應用對應時間步的噪聲混合，然後通過 U-Net 模型預測該噪聲，最後我的 Loss function 是使用均方誤差 (MSE) 來衡量實際噪聲 ϵ 和由 U-Net 預測的噪聲 $\epsilon^{\wedge}$ 之間的差異，讓模型學習如何逆轉 noise，並優化 U-Net 使其能夠更準確地預測噪聲。

(原本 Loss function 有做一個 L1 norm 的設定，但後來成果並沒有明顯的增長)

```
def sample(self, cond, size, device):
    """sample initial noise and generate images based on conditions"""
    n_sample = len(cond)
    x_i = torch.randn(n_sample, *size).to(device)
    sqrt_beta_t = torch.sqrt(self.betas).to(device)
    samples = []

    for idx in range(self.n_T-1, -1, -1):
        timestep = torch.tensor([idx / self.n_T]).to(device)
        z = torch.randn(n_sample, *size).to(device) if idx > 1 else 0
        eps = self.unet_model(x_i, cond, timestep)

        oneover_sqrt_alpha_t = self.sqrt_recip_alphas[idx] * (x_i - eps * self.mab_over_sqrt_alpha[idx])
        beta_t = sqrt_beta_t[idx] * z
        x_i = oneover_sqrt_alpha_t + beta_t

        if idx % 100 == 0:
            samples.append(x_i[9].clone())

    return x_i, samples
```

這是 DDPM 中的 Reverse process，它從完全隨機的噪聲開始，逐步逆向擴散過程生成目標圖像，公式大概如下：

$$\mathbb{E}_q \left[\underbrace{D_{\text{KL}}(q(\mathbf{x}_T|\mathbf{x}_0) \parallel p(\mathbf{x}_T))}_{L_T} + \underbrace{\sum_{t>1} D_{\text{KL}}(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) \parallel p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t))}_{L_{t-1}} - \underbrace{\log p_\theta(\mathbf{x}_0|\mathbf{x}_1)}_{L_0} \right]$$

$$p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \boldsymbol{\mu}_\theta(\mathbf{x}_t, t), \sigma_t^2 \mathbf{I})$$

$$\boldsymbol{\mu}_\theta(\mathbf{x}_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \right)$$

若想取得 $\mathbf{x}_{t-1}$ ，只需要套用推導出來的 reverse process 的公式，將 $\mathbf{x}_t$ 減去模型預測出來的雜訊並乘上係數，最終再加上雜訊 z 就可以取得了。因此，在每一步，使用 U-net 模型預測噪聲，然後根據預測的噪聲調整當前圖像狀態，逐步接近原始圖像，這個過程通過多次迭代逐漸細化圖像，最終產生高質量的生成圖像。

3. Train-Test 過程

```
class DDPM_TrainTest():
    def __init__(self, args, ddpm):
        super(DDPM_TrainTest).__init__()

        self.args = args
        self.lr = args.lr
        self.n_epoch = args.num_epochs
        self.device = args.device
        self.noise_type = args.noise_type
        self.sample_method = args.sample_method
        self.ddpm = ddpm
        self.evaluator = evaluation_model()

def train_ddpm(self, train_loader, test_loader, new_test_loader):
    optimizer = optim.Adam(self.ddpm.parameters(), lr=args.lr)
    lr_scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='max', factor=0.95, patience=3, min_lr=0)
    best_test_score, best_new_test_score = 0, 0

    Losses = []
    Test_Scores, New_Test_Scores = [], []

    for epoch in tqdm(range(args.num_epochs)):
        self.ddpm.train()
        optimizer.param_groups[0]['lr'] = args.lr * (1 - epoch / args.num_epochs) # linear lr decay

        for images, labels in tqdm(train_loader):
            optimizer.zero_grad()
            images = images.to(args.device)
            labels = labels.to(args.device)
            loss = self.ddpm(images, labels)
            loss.backward()
            optimizer.step()

        """testing"""
        test_score, grid, grid_denoise = self.test_ddpm(test_loader)
        path = os.path.join(args.test_result_path, self.sample_method, self.noise_type, f"test_{epoch}.png")
        os.makedirs(os.path.dirname(path), exist_ok=True)
        save_image(grid, path)

        save_path = os.path.join(self.args.denoise_path, self.noise_type, f"denoise_process_{epoch}.png")
        os.makedirs(os.path.dirname(save_path), exist_ok=True)
        save_image(grid_denoise, save_path)

        """new testing"""
        new_test_score, grid, _ = self.test_ddpm(new_test_loader)
        path = os.path.join(args.new_test_result_path, self.sample_method, self.noise_type, f"test_{epoch}.png")
        os.makedirs(os.path.dirname(path), exist_ok=True)
        save_image(grid, path)
```



```

        if test_score > best_test_score or new_test_score > best_new_test_score:
            if test_score > best_test_score:
                best_test_score = test_score
            else:
                best_new_test_score = new_test_score

        path = os.path.join([args.model_path, self.sample_method, self.noise_type,
                             f"ddpm_{epoch}_t{test_score:.3f}_nt{new_test_score:.3f}.pth"])
        os.makedirs(os.path.dirname(path), exist_ok=True)
        torch.save(self.ddpm.state_dict(), path)
        print("saved best test model")

    lr_scheduler.step(test_score)

    Losses.append(loss.item())
    Test_Scores.append(test_score)
    New_Test_Scores.append(new_test_score)

    print(f'Epoch: {epoch} train loss: {loss.item()} test score: {test_score} new_test_score: {new_test_score}')
    print()

return Losses, Test_Scores, New_Test_Scores

```

在 train_ddpm 中實作 DDPM 模型的訓練循環，訓練過程中包含配置 optimizer (採用 Adam) 和 Learning Scheduler (採用 ReduceLR)，這有助於在訓練過程中適應學習率以改善模型學習效果。訓練每一周期中，對於訓練數據集中的每一個 Batch 的數據，進行前向傳播，以及損失計算、反向傳播和優化步驟。

在每個 Epoch 訓練周期後，會對測試 test dataset 和 new test dataset 進行評估，評估是通過 test_ddpm 方法完成的 (會在下面做講解)，該方法在評估模式下生成圖像，並使用評估模型計算生成圖像的質量分數。最終，test 和 new test 的測試分數和生成的圖像都會被記錄和返回。

```

def test_ddpm(self, test_loader):
    self.ddpm.eval()
    x_gen, label, sample = [], [], []
    with torch.no_grad():
        for i, cond in enumerate(tqdm(test_loader)):
            cond = cond.to(self.device)
            size = (3, 64, 64)
            generated_images, samples = self.ddpm.sample(cond, size, self.device)

            x_gen.append(generated_images)
            sample.append(samples)
            label.append(cond)

        sample = torch.stack(samples, dim=0).squeeze()
        x_gen = torch.stack(x_gen, dim=0).squeeze().view(-1, 3, 64, 64) # [32, 3, 64, 64]
        label = torch.stack(label, dim=0).squeeze().view(-1, 24) # [32, 24]

        score = self.evaluator.eval(x_gen, label)

        grid = make_grid(x_gen, nrow=8, normalize=True)
        grid_denoise = make_grid(sample, nrow=10, normalize=True)

    return score, grid, grid_denoise

```

對於測試加載器中的每個 batch (條件 cond)，該條件通常包括目標標籤，並放入 DDPM 中的 sample 產出 denoise 後的圖像以及 denoise 過程的圖像集，並將生成的圖像和圖像集分別添加到 x_gen 和 sample 列表中，將條件添加到 label 列表中。

然後使用 torch.stack() 將列表中的數據合併成一個新的 tensor，並使用 ResNet18 架構的評估模型來評估生成的圖像相對於它們的條件的質量。

最後將生成的圖像和圖像集視覺化 (使用 make_grid 函數創建一個圖像網格)，並返回生成圖像的評分 (score)、生成圖像的網格 (grid)、以及生成過程中樣本的網格 (grid_denoise)。

c. Conditional GAN

這次實驗我使用的 GAN 架構是 Deep Convolutional GAN (DCGAN), 主要分成 Discriminator, Generator 架構, 以及 Train-Test三個部份, 下面會詳細介紹每個部份的原理和架構。

1. Generator 架構

```
class GAN_Generator(nn.Module):
    def __init__(self, dim_z, dim_c):
        super(GAN_Generator, self).__init__()

        self.dim_z = dim_z
        self.dim_c = dim_c
        self.conditionExpand=nn.Sequential(
            nn.Linear(24, dim_c),
            nn.ReLU(0.2)
        )
        layers = [
            (dim_z + dim_c, 512, 4, 1, 0),
            (512, 256, 4, 2, 1),
            (256, 128, 4, 2, 1),
            (128, 64, 4, 2, 1),
            (64, 3, 4, 2, 1)
        ]
        net_layers = []
        for i, (in_channels, out_channels, kernel_size, stride, padding) in enumerate(layers):
            net_layers.append(nn.ConvTranspose2d(in_channels, out_channels, kernel_size, stride, padding, bias=False))
            if i < len(layers) - 1: # Add BatchNorm and ReLU to all but the last layer
                net_layers.append(nn.BatchNorm2d(out_channels))
                net_layers.append(nn.ReLU(True))
            net_layers.append(nn.Tanh()) # Add Tanh at the end

        self.net = nn.Sequential(*net_layers)

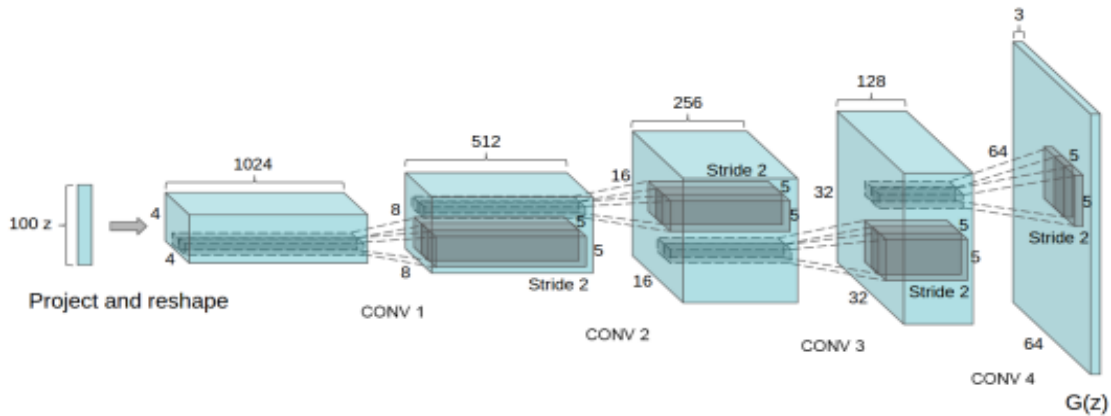
    def forward(self, x, label):
        x = x.view(-1, self.dim_z, 1, 1)
        label = self.conditionExpand(label).view(-1, self.dim_c, 1, 1)
        out = torch.cat((x, label), 1)
        return self.net(out)

    def initialize_weights(self):
        for m in self._modules:
            if isinstance(self._modules[m], nn.ConvTranspose2d) or isinstance(self._modules[m], nn.Conv2d):
                self._modules[m].weight.data.normal_(0, 0.02)
                self._modules[m].bias.data.zero_()
```

在 GAN_Generator 中, 一開始會用 initialize_weights 方法負責初始化網路中所有捲積層的權重。

而 Generator 的結構設計用於接受隨機雜訊和 Condition label 作為輸入, 並輸出合成影像。首先設定了 Generator 的基本參數, 包括雜訊維度 dim_z 和 label 維度 dim_c。這些維度決定了輸入雜訊和 Condition label 的大小。接著定義了一個序列模型 conditionExpand, 這個模型透過一個線性層和 ReLU 激活函數擴展 label 的維度。

而主體是由多個反捲積層(ConvTranspose2d)所組成的網路(如下圖), 這些層透過逐步上取樣和增加通道的維度, 將輸入的低維向量轉換成高維度影像資料。每一層反捲積後通常跟隨一個批歸一化(BatchNorm2d)和ReLU 激活函數, 除了最後一層外, 最後一層使用的是Tanh 激活函數, 將輸出值正規化到[-1, 1] 範圍內。



在 forward 方法中，首先將輸入的隨機雜訊 x 和 label 處理成適當的形狀，使其能夠透過網路處理，然後將擴展後的 label 和雜訊向量在通道維度上進行拼接，而這個拼接後的張量透過 Generator 產生最終的影像。

2. Discriminator 架構

```
class GAN_Discriminator(nn.Module):
    def __init__(self, num_classes, image_size):
        super(GAN_Discriminator, self).__init__()
        self.image_size = image_size
        self.conditionExpand = nn.Sequential(
            nn.Linear(num_classes, image_size * image_size),
            nn.LeakyReLU(0.2, True)
        )

        layers = [
            (4, 64, 4, 2, 1),      # (3+1) x 64 x 64
            (64, 128, 4, 2, 1),    # (64) x 32 x 32
            (128, 256, 4, 2, 1),   # (128) x 16 x 16
            (256, 512, 4, 2, 1),   # (256) x 8 x 8
            (512, 1, 4, 1, 0)      # (512) x 4 x 4
        ]

        net_layers = []
        for i, (in_channels, out_channels, kernel_size, stride, padding) in enumerate(layers):
            net_layers.append(nn.Conv2d(in_channels, out_channels, kernel_size, stride, padding, bias=False))
            if i == 0:
                net_layers.append(nn.LeakyReLU(0.2, inplace=True))
            if i < len(layers) - 1 and i != 0: # Add BatchNorm and LeakyReLU to all but the last layer
                net_layers.append(nn.BatchNorm2d(out_channels))
                net_layers.append(nn.LeakyReLU(0.2, inplace=True))
        #net_layers.append(nn.Sigmoid())

        self.net = nn.Sequential(*net_layers)

    def forward(self, x, label):
        label = self.conditionExpand(label).view(-1, 1, 64, 64)
        out = torch.cat((x, label), 1)
        return self.net(out).view(-1, 1)

    def initialize_weights(self):
        for m in self._modules:
            if isinstance(self._modules[m], nn.ConvTranspose2d) or isinstance(self._modules[m], nn.Conv2d):
                self._modules[m].weight.data.normal_(0, 0.02)
                self._modules[m].bias.data.zero_()
```

GAN_Discriminator 的結構設計用於接受 Generator 產出的合成影像，以及其 Condition label 作為輸入，為了能夠評估生成圖像的真實性，並透過學習區分合成圖像與真實圖像，從而在訓練過程中指導 Generator 產生越來越真實的圖像。

Discriminator 的主體是由多個 Conv2d 所組成的結構，這些層透過逐步降低影像的空間維度和增加通道的維度，將輸入的高維度影像資料轉換成機率值，表示輸入影像是真實影像的機率。每一層卷積後通常跟隨一個 LeakyReLU 激活函數，用於保持梯度資訊並防止神經元死亡，第一層之後和最後一層之前的捲積層也會加入 BatchNorm2d，以穩定訓練過程。

而最後一層原本是要加上 sigmoid 使 noise 輸出壓縮到0和1之間的範圍內，然而經由實驗發現，加上後生成效果反而變得不好，noise 分佈較為雜亂，因此在這部份我將它拿掉。

在 forward 中，abel 經過處理後與輸入影像在通道維度上進行拼接，形成 Discriminator 的輸入，合併後的張量透過定義好的捲積網路來處理，最後輸出一個機率值，表示輸入影像為真實影像的可能性。

3. Train-Test 過程

```
class GAN_TrainTest():
    def __init__(self, args, generator, discriminator):
        super(GAN_TrainTest).__init__()

        self.args = args
        self.n_epoch = args.num_epochs
        self.device = args.device
        self.generator = generator
        self.discriminator = discriminator
        self.evaluator = evaluation_model()
        self.dim_z = args.dim_z
        self.dim_c = args.dim_c
```

在 GAN_TrainTest 中，為 GAN 的訓練以及測試過程，整個過程包良兩個步驟，分別是 Generator 和 Discriminator 的交替訓練以及性能評估的過程。

這種交替訓練有助於保持 Generator 和 Discriminator 之間的平衡，防止一方在訓練過程中過度主導，導致訓練失敗。

```

def train_gan(self, train_loader, test_loader, new_test_loader):
    optimizer_g = optim.Adam(self.generator.parameters(), lr=args.lr_generator, betas=(0.5, 0.999))
    optimizer_d = optim.Adam(self.discriminator.parameters(), lr=args.lr_discriminator, betas=(0.5, 0.999))

    lr_scheduler_g = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer_g, mode='max', factor=0.95, patience=5, min_lr=0)
    lr_scheduler_d = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer_d, mode='max', factor=0.95, patience=5, min_lr=0)

    criterion = nn.BCEWithLogitsLoss()
    best_test_score, best_new_test_score = 0, 0

    Generator_Losses, Discriminator_Losses = [], []
    Test_Scores, New_Test_Scores = [], []

    for epoch in tqdm(range(args.num_epochs)):
        generator_loss, discriminator_loss = 0, 0

        for images, labels in tqdm(train_loader):
            self.generator.train()
            self.discriminator.train()

            images = images.to(args.device)
            labels = labels.to(args.device)
            batch_size = images.size(0)

            # Train Discriminator
            noise = torch.randn(batch_size, args.dim_z, 1, 1, device=args.device)
            fake_images = self.generator(noise, labels)

            real_outputs = self.discriminator(images.detach(), labels)
            fake_outputs = self.discriminator(fake_images.detach(), labels)

            real_label = torch.ones_like(real_outputs, dtype=torch.float)
            fake_label = torch.zeros_like(fake_outputs, dtype=torch.float)

            loss_real = criterion(real_outputs, real_label)
            loss_fake = criterion(fake_outputs, fake_label)
            loss_d = loss_real + loss_fake

            optimizer_d.zero_grad()
            loss_d.backward()
            optimizer_d.step()

```

訓練 Discriminator 時會同時使用真實影像 (real_image) 和產生的假影像 (fake_image), 首先將真實的影像 (也就是 train image) 輸入到 Discriminator 並評估這些影像, 輸出一個機率值, 表示每張影像是真實影像的機率, 這個機率值會與標籤1(真實)比較, 計算真實影像損失 (loss_real)。

再來請 Generator 接收隨機雜訊作為輸入, 產生假影像並輸入到 Discriminator, 並且同樣評估這些假影像, 並輸出一個機率值, 表示影像是真實的機率。這個機率值與標籤0(假)比較, 計算假影像損失 (loss_fake)。

最後將 loss_real 和 loss_fake 相加, 使用這個總損失透過反向傳播更新 Discriminator。

```

# -----
# Train Generator for 3 times
for i in range(3):
    noise = torch.randn(batch_size, args.dim_z, 1, 1, device=args.device)
    fake_image_test = self.generator(noise, labels)

    outputs = self.discriminator(fake_image_test, labels)
    generator_label = torch.ones_like(outputs, dtype=torch.float)

    loss_g = criterion(outputs, generator_label)

    optimizer_g.zero_grad()
    loss_g.backward()
    optimizer_g.step()

discriminator_loss += loss_d.item()
generator_loss += loss_g.item()

```

為了讓 Generator 的目標是欺騙 Discriminator，使其將生成的假圖像判定為真實圖像，因此需要訓練 Generator 從隨機雜訊產生影像，並將這些假影像被送入 Discriminator 進行評估。

計算損失與 Discriminator 的假影像處理類似，但在這裡的目標是使 Discriminator 的輸出接近 1 (真實)，並將計算出的損失將用來更新 Generator，鼓勵其產生越來越難以被 Discriminator 區分的圖像，再使用從 Discriminator 獲得的回饋 (即鑑別器認為生成圖像的真實性) 來更新生成器的權重。

而在本次 GAN 的實作中，一開始我讓 Generator 和 Discriminator 各訓練一次，然而實驗過一次，發現 Generator 相較於 Discriminator 較難以訓練 (例如，Generator 太弱，生成的圖像品質低)，因此我設定讓 Generator 的更新步驟比 Discriminator 的多。

```
def test_gan(self, test_loader):
    self.generator.eval()
    self.discriminator.eval()
    x_gen = []
    scores = 0
    with torch.no_grad():
        for labels in tqdm(test_loader):
            labels = labels.to(args.device)
            batch_size = labels.size(0)
            noise = torch.randn(batch_size, args.dim_z, 1, 1, device=args.device)
            fake_image = self.generator(noise, labels)
            x_gen.append(fake_image)
            score = self.evaluator.eval(fake_image, labels)
            scores += score

    x_gen = torch.stack(x_gen, dim=0).squeeze().view(-1, 3, 64, 64) # [32, 3, 64, 64]
    grid = make_grid(x_gen, nrow=8, normalize=True)

    return scores, grid
```

```
"""testing"""
test_score, grid = self.test_gan(test_loader)
path = os.path.join(args.test_result_path, f"test_{epoch}.png")
os.makedirs(os.path.dirname(path), exist_ok=True)
save_image(grid, path)

new_test_score, grid = self.test_gan(new_test_loader)
path = os.path.join(args.new_test_result_path, f"test_{epoch}.png")
os.makedirs(os.path.dirname(path), exist_ok=True)
save_image(grid, path)

if test_score > best_test_score or new_test_score > best_new_test_score:
    if test_score > best_test_score:
        best_test_score = test_score
    else:
        best_new_test_score = new_test_score

    generator_path = os.path.join(args.model_path, 'g', f"epoch{epoch}_t{test_score:.3f}_nt{new_test_score:.3f}.pth")
    os.makedirs(os.path.dirname(generator_path), exist_ok=True)
    torch.save(self.generator.state_dict(), generator_path)

    discriminator_path = os.path.join(args.model_path, 'd', f"epoch{epoch}_t{test_score:.3f}_nt{new_test_score:.3f}.pth")
    os.makedirs(os.path.dirname(discriminator_path), exist_ok=True)
    torch.save(self.discriminator.state_dict(), discriminator_path)

    print("Saved best test model")

    lr_scheduler_d.step(test_score)
    lr_scheduler_g.step(test_score)
    generator_loss /= len(train_loader)
    discriminator_loss /= len(train_loader)

    Generator_Losses.append(generator_loss)
    Discriminator_Losses.append(discriminator_loss)
    Test_Scores.append(test_score)
    New_Test_Scores.append(new_test_score)

    print(f'Epoch: {epoch} train generator loss: {generator_loss} train discriminator loss: {discriminator_loss}
          test score: {test_score} new_test_score: {new_test_score}')
    print()

return Generator_Losses, Discriminator_Losses, Test_Scores, New_Test_Scores
```


跟 DDPM 測試時的步驟一樣，在每個 Epoch 訓練周期後，會對測試 test dataset 和 new test dataset 進行評估，評估是通過 test_ddpm 方法完成的 (會在下面做講解)，該方法在評估模式下生成圖像，並使用評估模型計算生成圖像的質量分數。最終，test 和 new test 的測試分數和生成的圖像都會被記錄和返回，其中這個 Epoch 的訓練權重也會分別存下來。

3. Results and discussion

a. Show synthetic image grids and a denoising process image

i. Conditional DDPM

訓練時的參數設定：

- learning rate = 0.0001
- batch size = 32
- Epoch = 100
- Optimizer = Adam
- learning scheduler = ReduceLR (patience = 5, factor = 0.95)

DDPM	Test	New test
linear	0.714 (At epoch 71)	0.738 (At epoch 87)
cosine	0.708 (At epoch 85)	0.75 (At epoch 82)

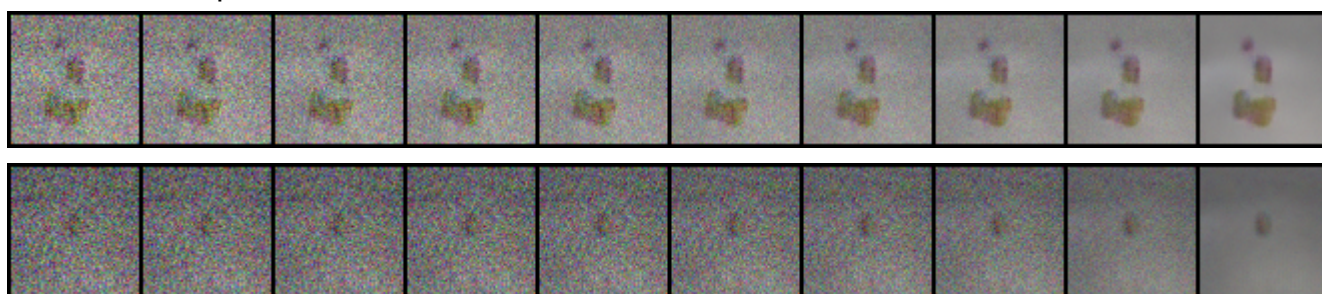
DDPM Denoise process image:

(上面 : linear, 下面 : cosine)

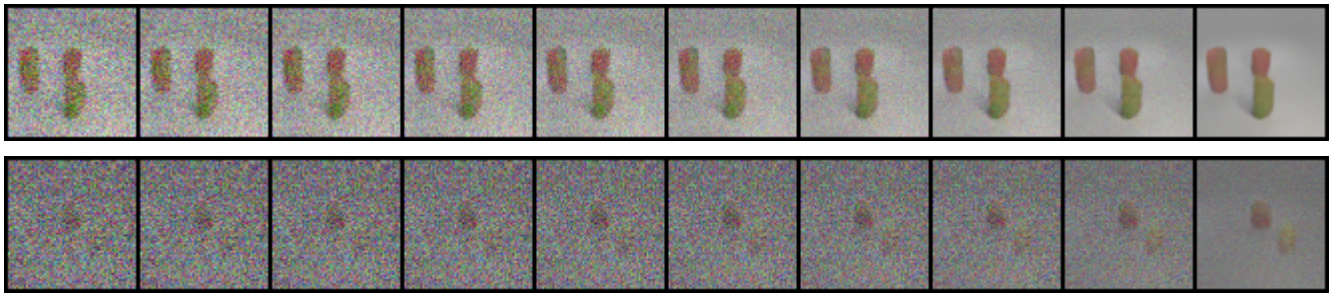
Epoch 0:



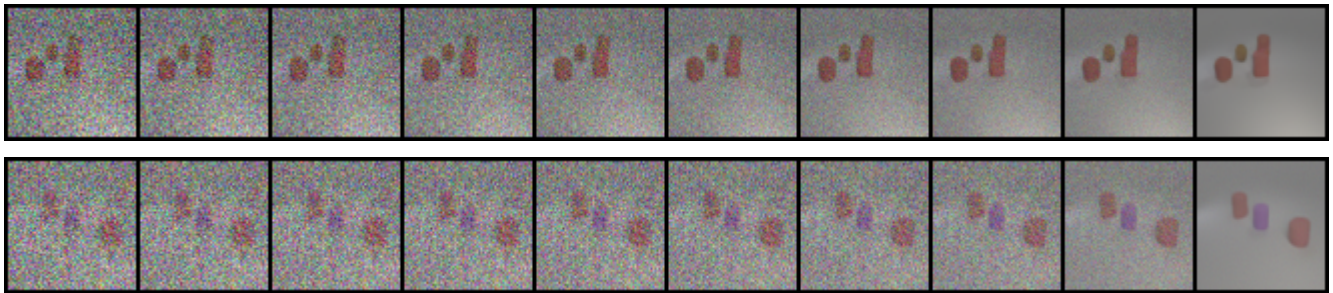
Epoch 5:



Epoch 10:

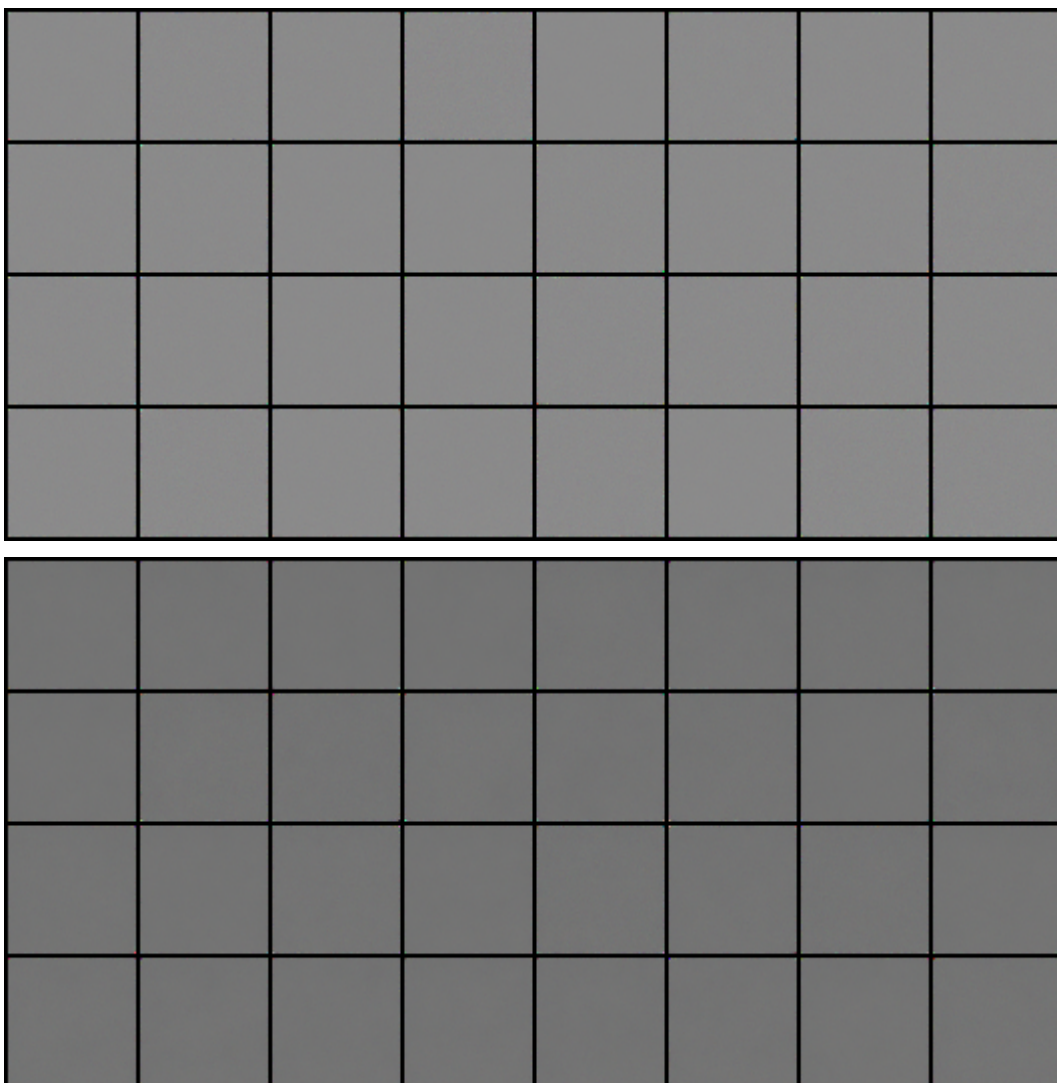


Epoch 50:

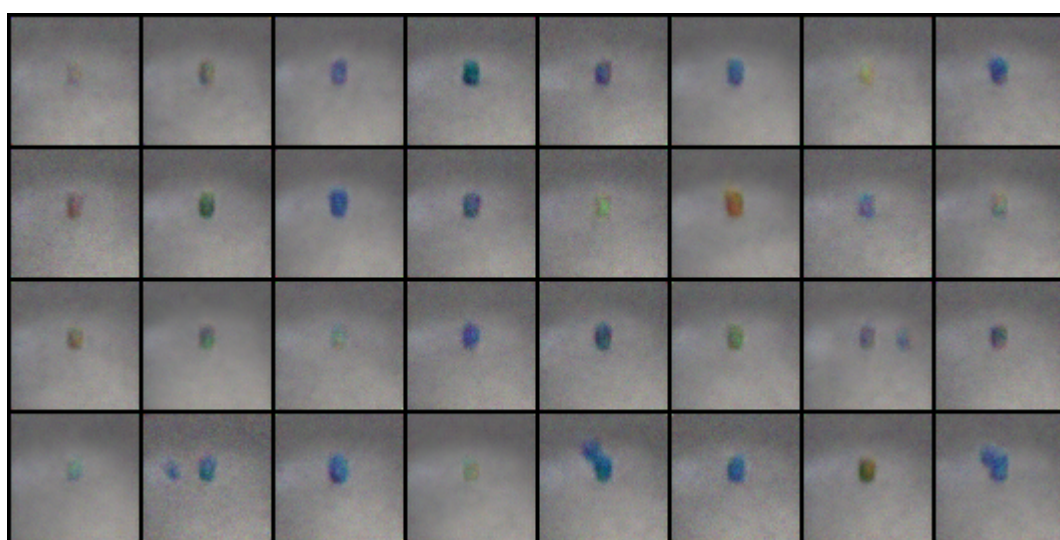
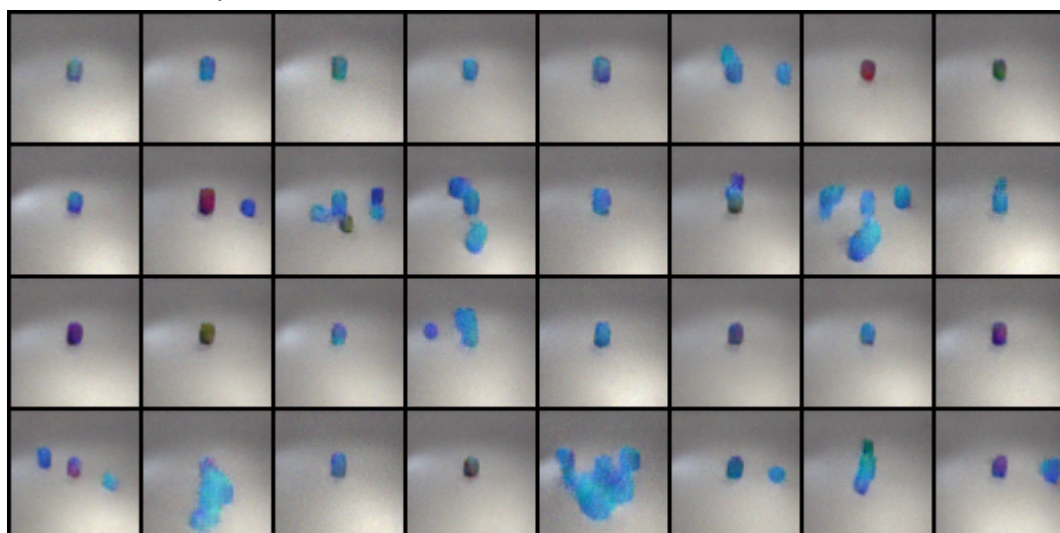


DDPM Test Grid image:

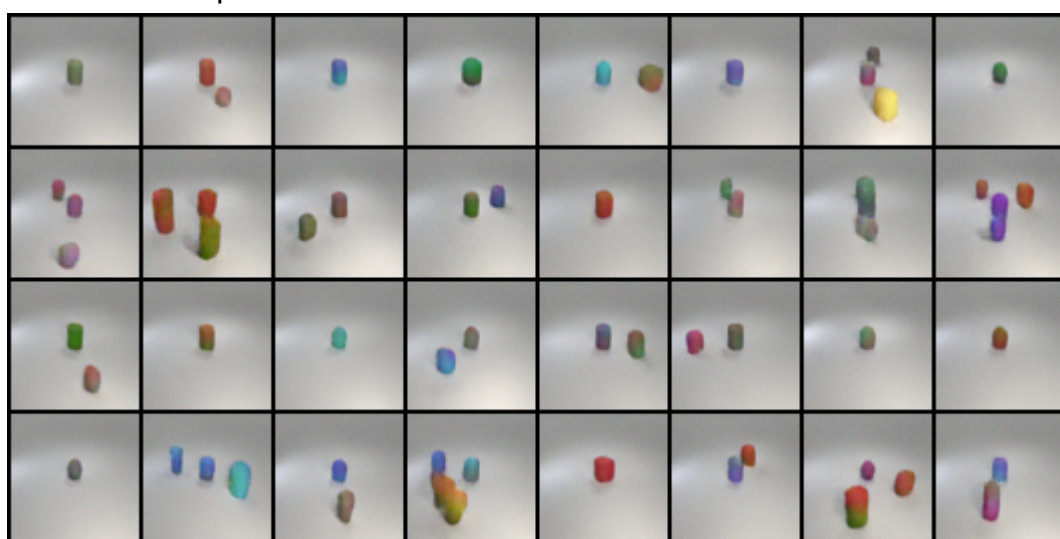
Epoch 0:

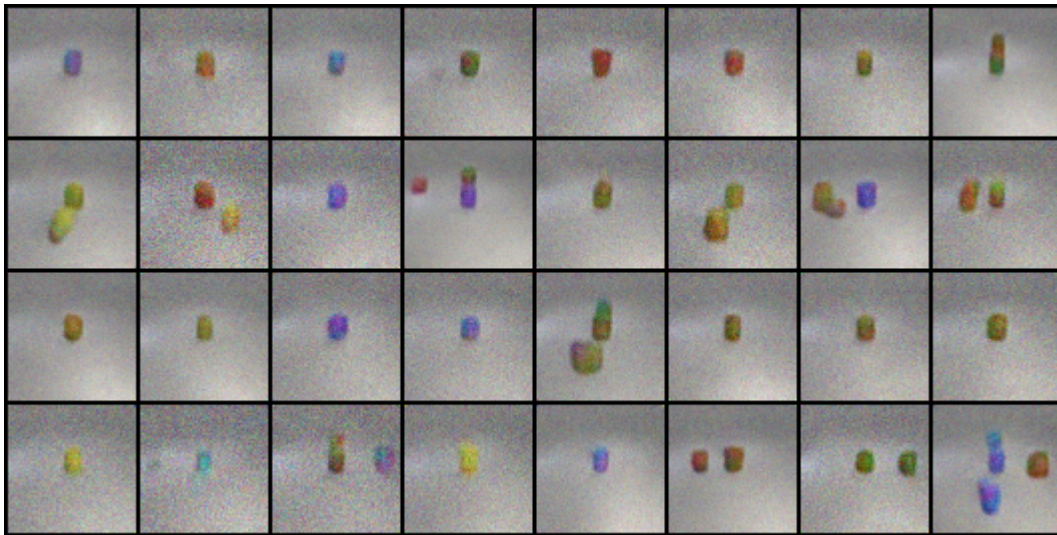


Epoch 5:

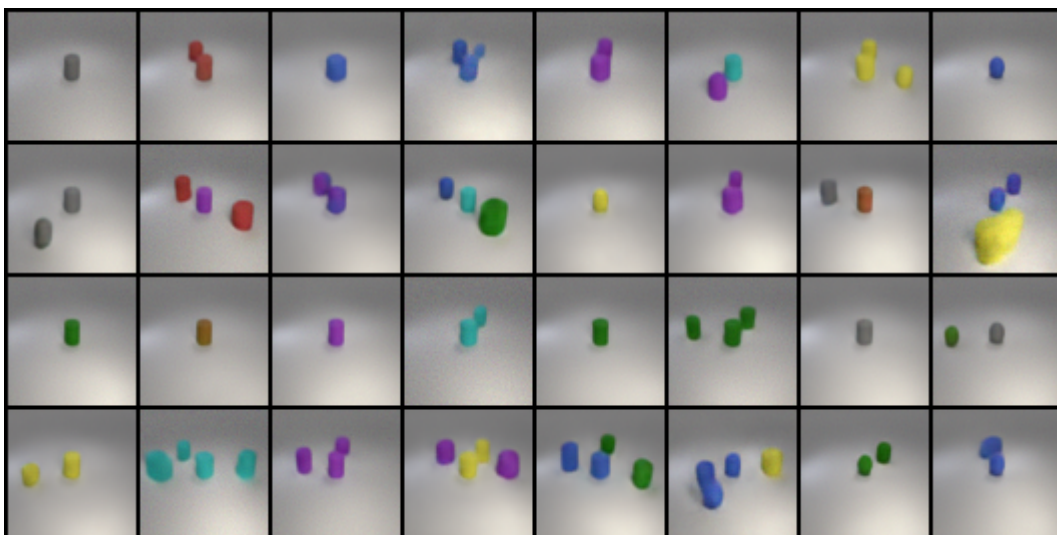
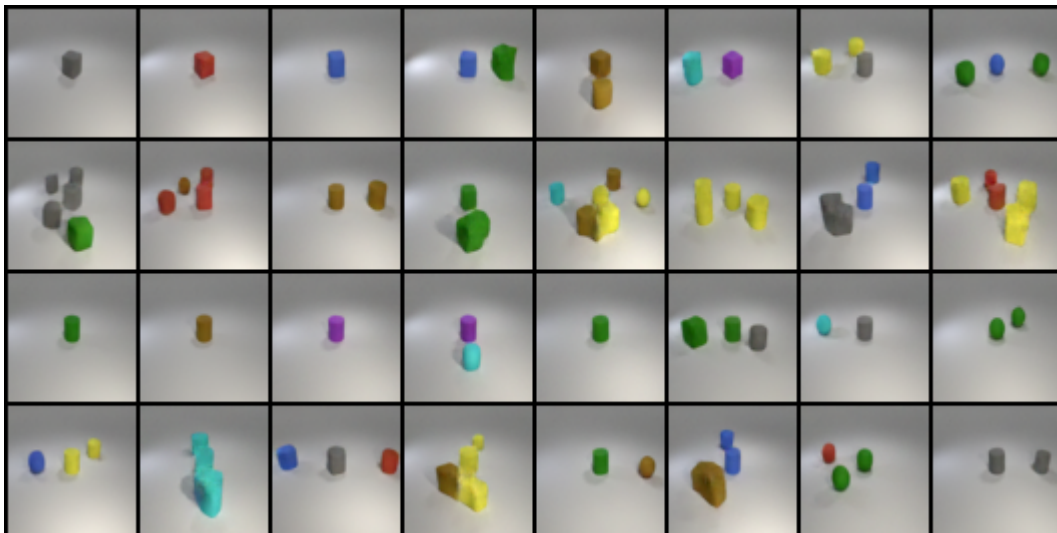


Epoch 10:



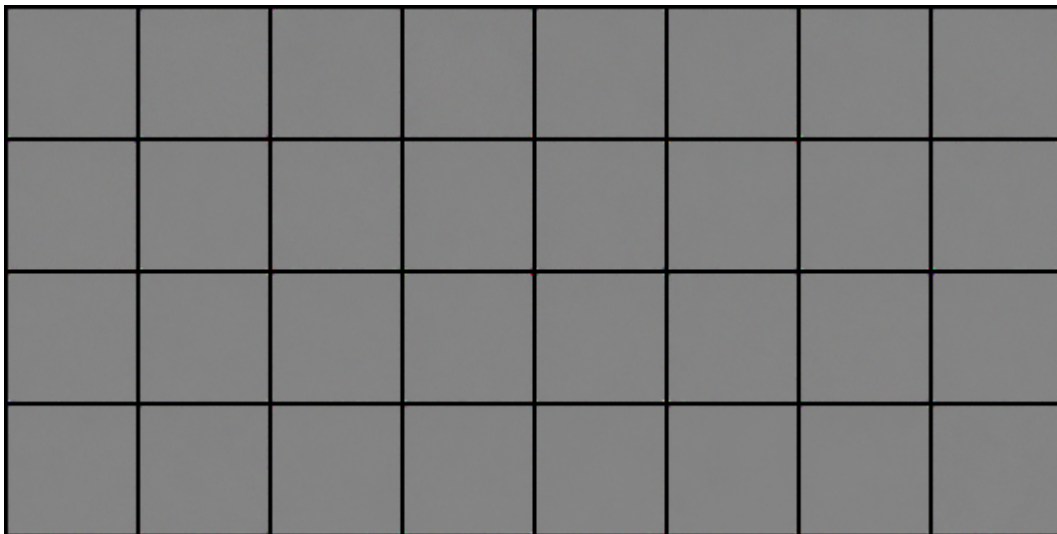


Epoch 50:

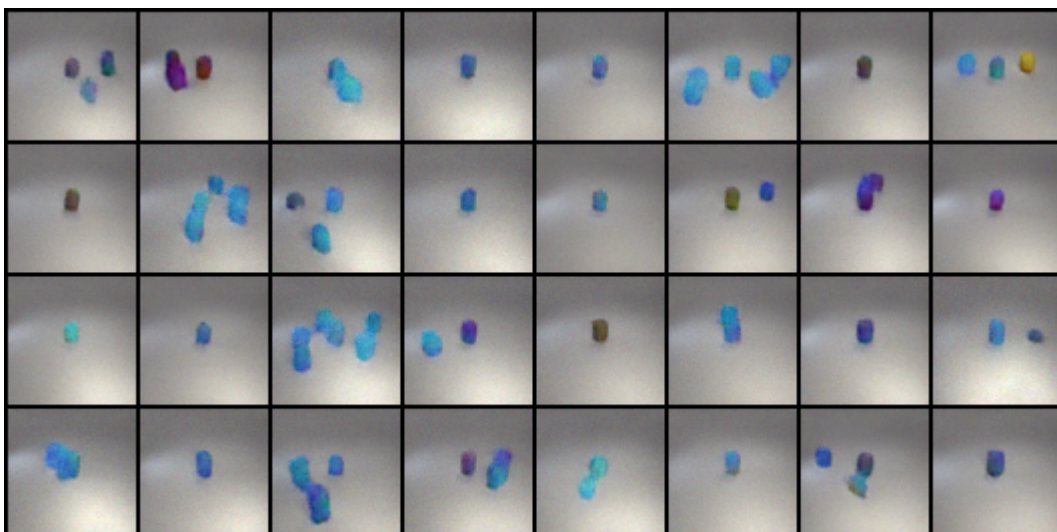


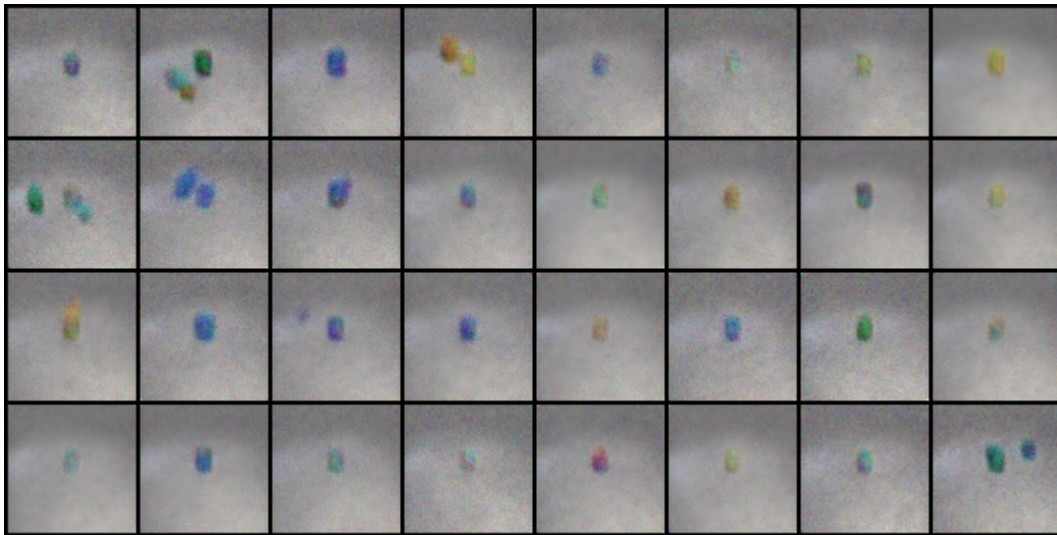
DDPM New Test Grid image:

Epoch 0:

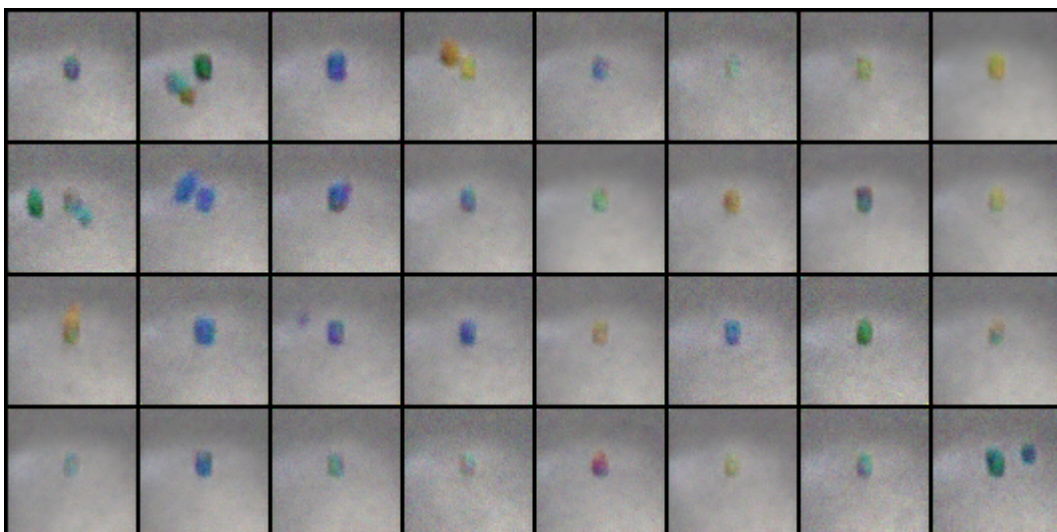
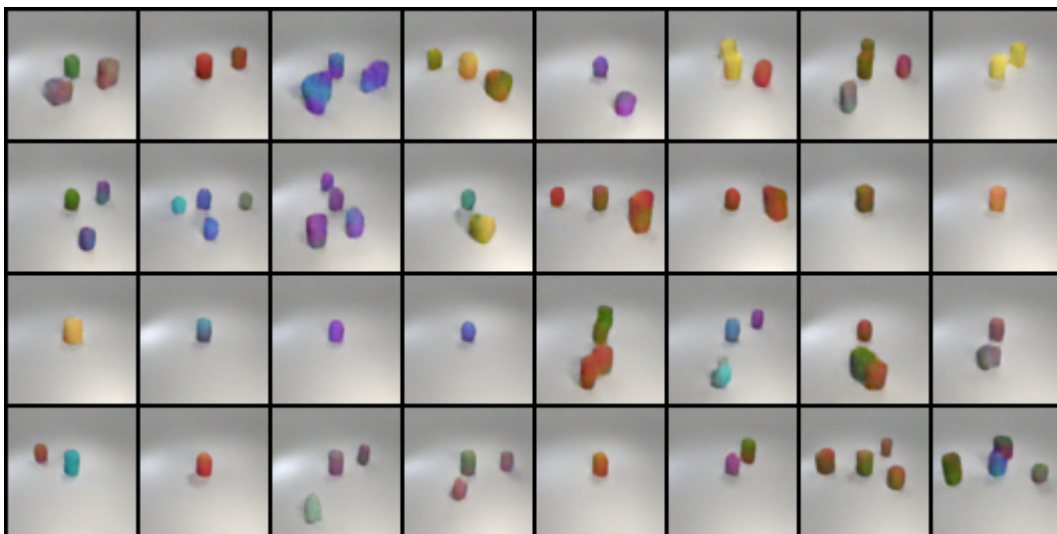


Epoch 5:

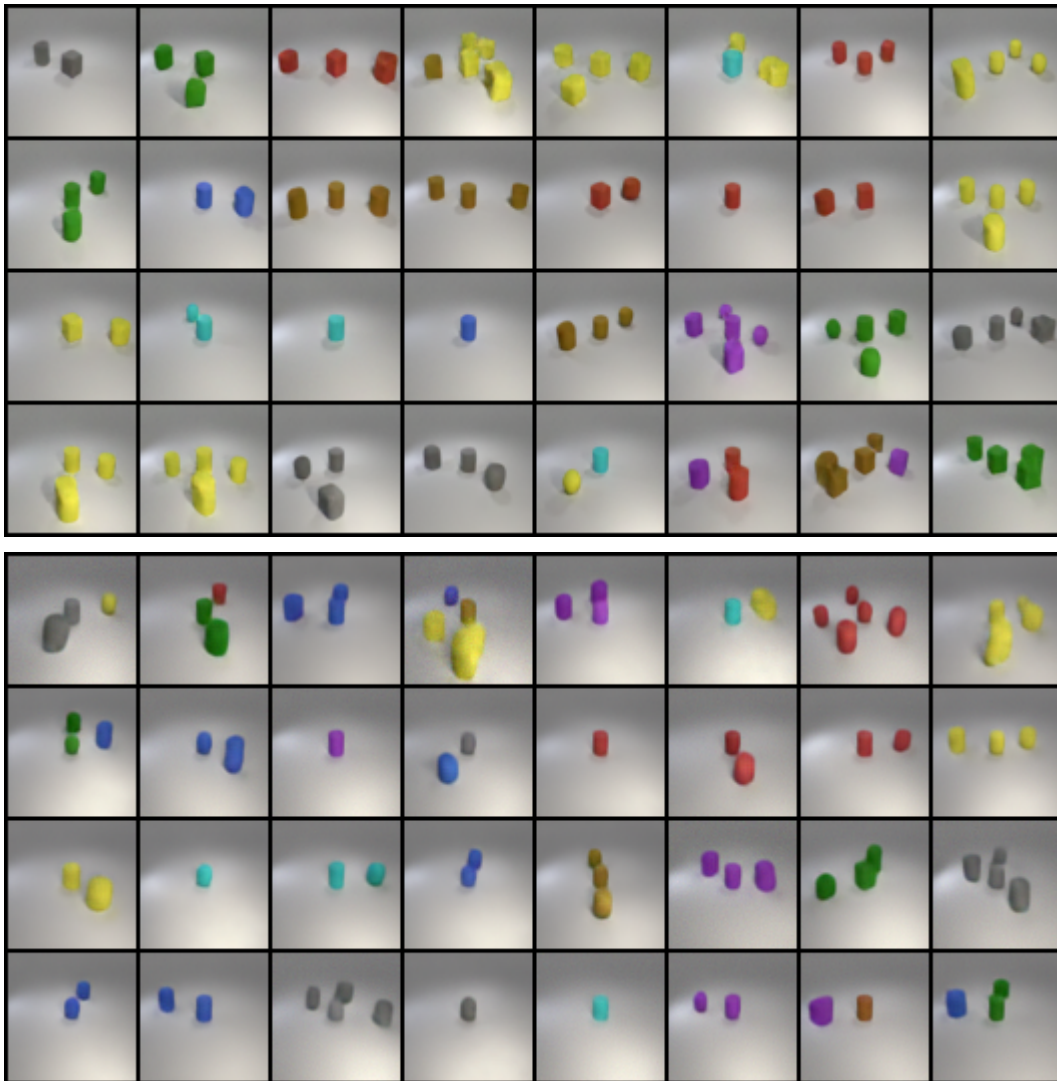




Epoch 10:



Epoch 50:



DDPM Conclusion:

由上面可以發現，不管是 Denoise process 和 Grid image，使用 linear 的 noise scheduler 方式會比使用 cosine 更早達到 denoise 的效果。原因可能是 linear 按照固定的速率從初始低噪聲水平直線增加至最高噪聲水平，並且在整個生成過程中保持噪聲增加的均勻性。

而 cosine 則是根據余弦函數的形狀調整噪聲水平，使噪聲的增加在初期較慢，中間階段較快，但我發現這種平滑的過渡有助於模型更有效地學習從低噪聲到高噪聲的過渡，尤其有利於生成過程中細節的保留和圖像質量的提升。因此，而由結果可以發現 cosine 到後期的表現比 linear 更好，保持圖像細節和紋理方面表現更為出色。

ii. Conditional GAN

訓練時的參數設定：

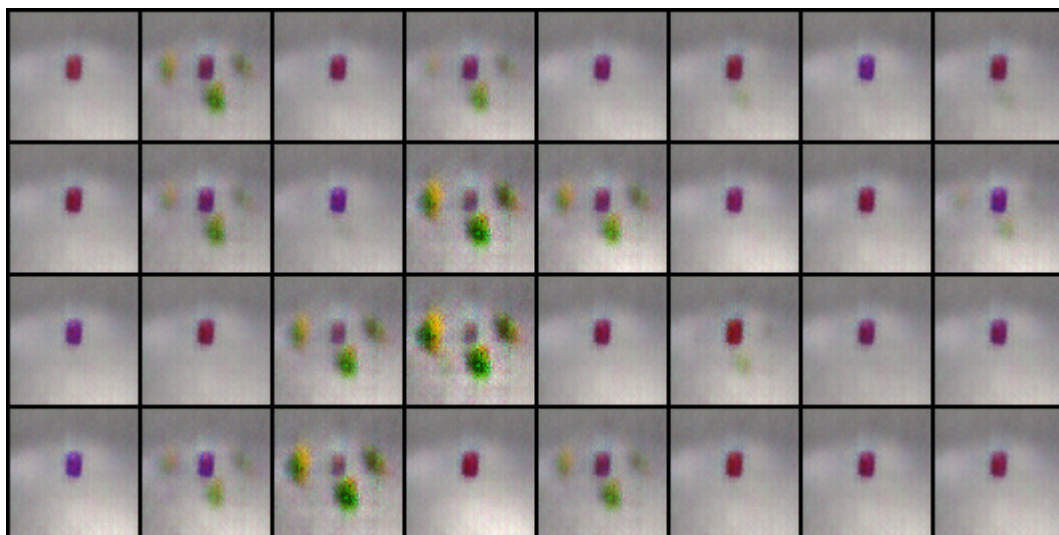
- Generator learning rate = 0.0002
- Discriminator learning rate = 0.0002
- batch size = 32
- Epoch = 300
- Optimizer = Adam (beta = [0.5, 0.999])
- learning scheduler = ReduceLR (patience = 5, factor = 0.95)

- $\text{dim_z} = 100; \text{dim_c} = 200$

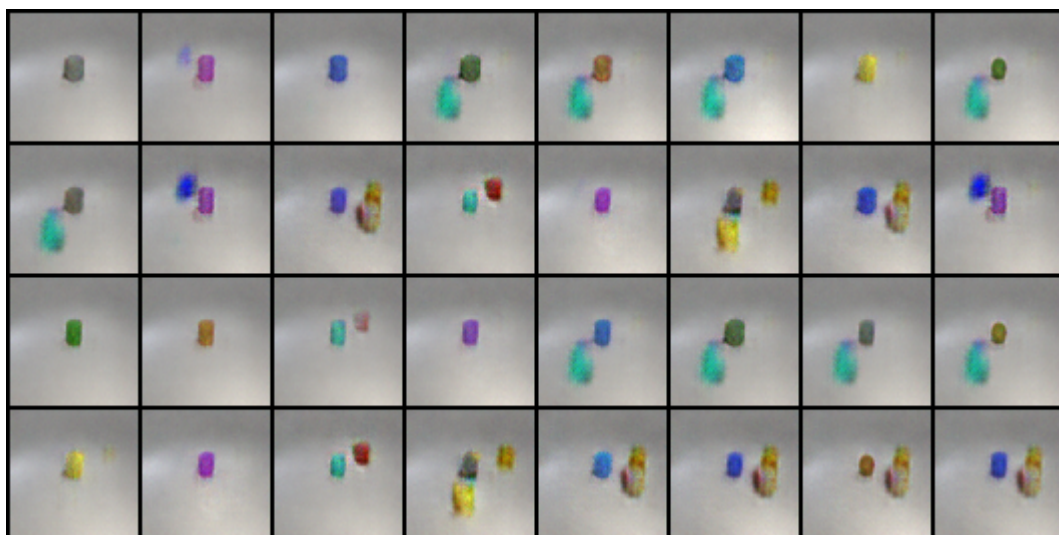
	Test	New test
GAN	0.68 (At epoch 115)	0.75 (At epoch 129)

GAN Test Grid image

Epoch 0:



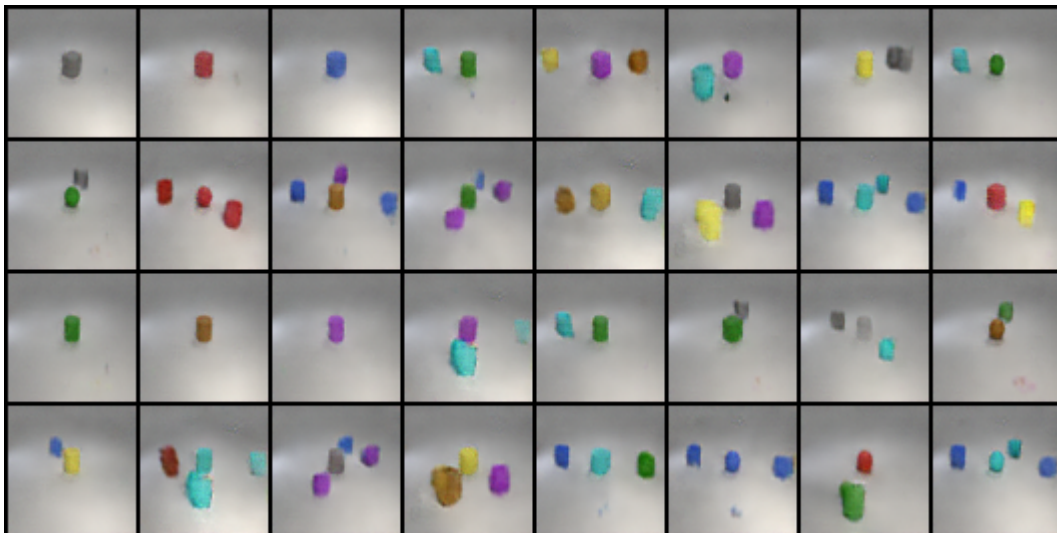
Epoch 5:



Epoch 10:

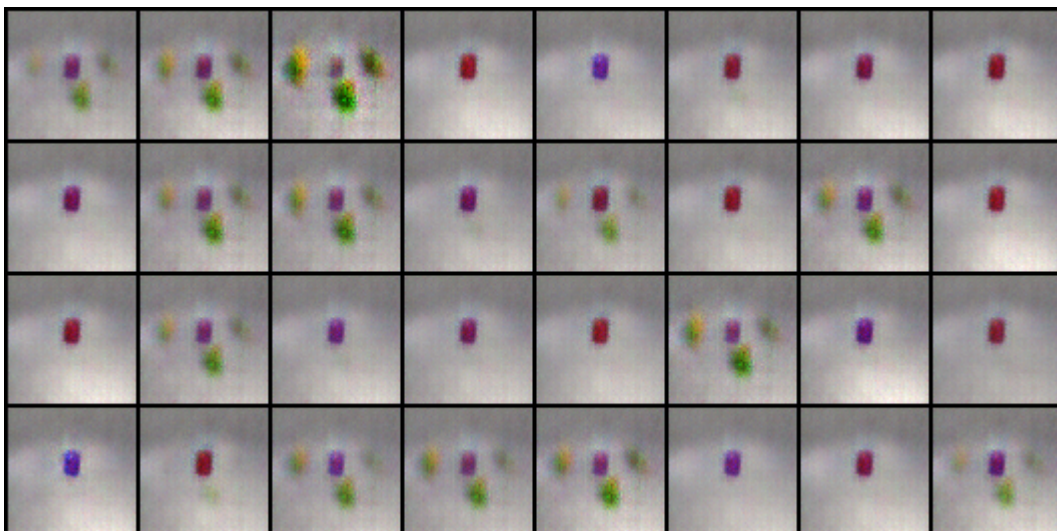


Epoch 50:

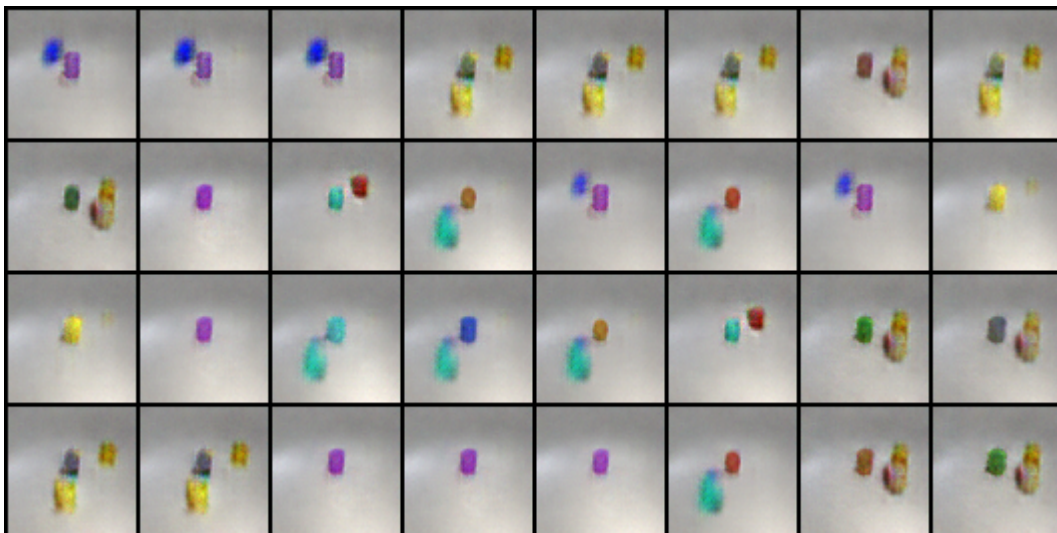


GAN New Test Grid image

Epoch 0:



Epoch 5:



Epoch 10:



Epoch 50:



GAN Conclusion:

由上面可以發現，雖然 GAN 生成的圖片在早期的表現比較好，但是圖像的細節比起 DDPM 較為粗糙，有可能是在訓練時它放大訓練數據中的偏差，因此生成的圖片也可能出現這些問題。

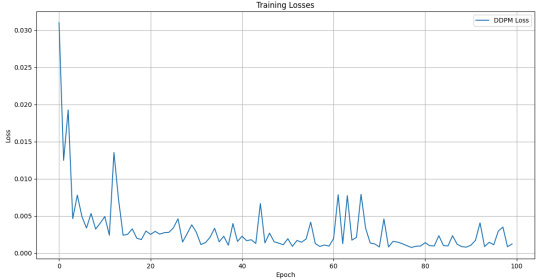
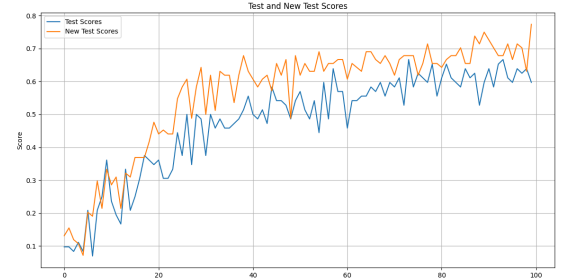
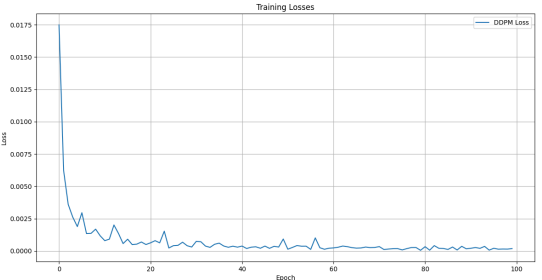
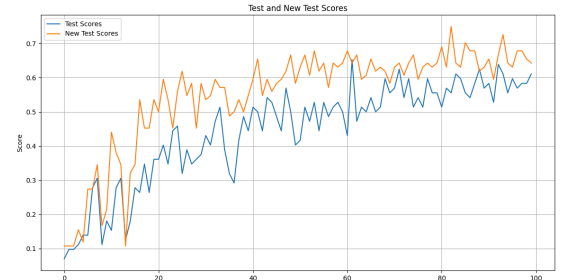
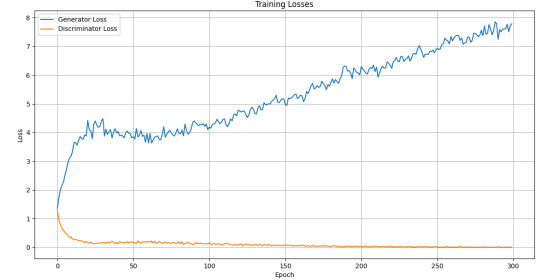
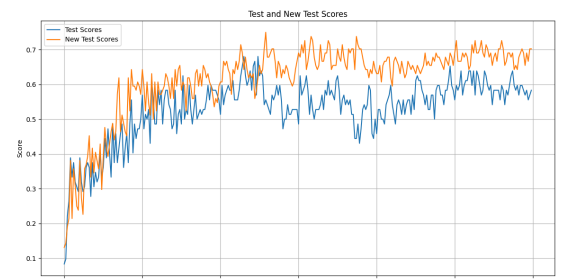
b. Compare the advantages and disadvantages of GAN and DDPM

	advantages	disadvantages
DDPM	- DDPM 的訓練過程相比於 GAN 更加穩定，且可以比 GAN 生成高質量的圖像，且細節和一致性通常比 GAN 生成的圖像更好。 - 比起 GAN，DDPM 訓練時的收斂性比較平穩，相較不容易出現訓練震盪。	DDPM 生成圖像需要多個步驟的擴散過程，速度較慢，所需的計算資源相對較高。
GAN	- 比起 DDPM，GAN 訓練較快，且 Generator 可以迅速生成新的圖像，幾乎不需等待，因此在相同時間內可以迭代更多次。 - 我自認為比起 DDPM，GAN 的架構較為直觀：就是兩個網路架構在彼此競爭，並且互相學習和適應對方的策略，因此如果未來要改善的話會比 DDPM 還要容易。	- GAN 的訓練通常是不穩定的，尤其是 Generator，即使在實驗中多次訓練它，但 loss 依舊逐漸上升，如下圖。 - GAN 沒有一個固定的指標來衡量生成圖像的質量，通常要依靠 Discriminator 的判斷能力。 - 根據文獻可知，GAN 較難在不同種類的圖像上學到複雜的分布，而由實驗可以知道 GAN 確實較難達到 0.7 的準確率。

c. Discussion of extra implementations or experiments

Train Loss

DDPM	Loss	Score
------	------	-------

linear		
cosine		
GAN	Loss	Score
		

由上表的比較可以發現，DDPM 的訓練比起 GAN 較為穩定，而且 loss 是有在逐漸下降，反而 GAN 的 Generator 最難訓練。其中觀察 DDPM 的訓練過成，發現 cosine 比起 linear 較為穩定，loss 不會起伏不定，反之 linear 比起 cosine 更早達到 0.7 的準確率。

而這次的實驗，我並沒有將 DDPM 和 GAN 設定同個Epoch (DDPM 為 100, GAN 為 300)，原因是 DDPM 訓練較慢，而觀察到 score 的趨勢後，如果將訓練次數拉長一點，有可能會持續成長。

Reference

- <https://arxiv.org/pdf/2006.11239>
- <https://arxiv.org/pdf/1511.06434>
- <https://adam-study-note.medium.com/diffusion-model-denoising-diffusion-probabilistic-models-ddpm-%E8%A9%B3%E7%B4%B0%E4%BB%8B%E7%B4%B9-5ce77b6b64d4>